

Proyecto final: Sistema de Gestión de Gimnasio

Objetivo de la aplicación

Nos han contactado para desarrollar una aplicación web completa. Dicha aplicación permitirá gestionar un gimnasio donde los usuarios podrán consultar clases disponibles, reservar plazas y gestionar sus reservas. Además, existirá un panel de administración para gestionar el funcionamiento del gimnasio.

El objetivo es desarrollar una aplicación funcional que simule el funcionamiento básico de un sistema real de gestión de gimnasio implementada en PHP y MySQL, donde:

- Los clientes puedan consultar clases y reservar plaza.
- Los entrenadores puedan ver las clases que imparten.
- Los administradores puedan gestionar usuarios, clases y reservas.

La aplicación debe diferenciar el acceso y las funcionalidades según el rol del usuario.

Descripción y requisitos del proyecto

Roles del sistema

El sistema tendrá tres tipos de usuarios:

- Administrador, el cual puede: gestionar usuarios, crear y gestionar clases, gestionar horarios, ver todas las reservas y acceder a estadísticas del gimnasio.
- Cliente, el cual puede: ver clases disponibles, reservar clases, cancelar reservas, ver su historial de reservas y editar su perfil.
- Entrenador, el cual puede: ver las clases que imparte, ver la lista de alumnos, inscritos en sus clases, consultar su horario.

Entidades principales

La aplicación deberá incluir al menos las siguientes entidades.

Usuarios (users): cada usuario tiene un rol que determina sus permisos en el sistema.

Campos mínimos:

- id
- nombre
- email
- password
- telefono
- rol (admin / cliente / entrenador)
- created_at
- updated_at

Clases (classes): representa las clases que se ofrecen en el gimnasio. Campos:

- id
- nombre
- descripcion
- duracion
- capacidad
- imagen
- entrenador_id
- created_at
- updated_at

Horarios (*schedules*): permite asignar diferentes horarios a una clase. Campos:

- id
- class_id
- fecha
- hora_inicio
- hora_fin
- created_at
- updated_at

Reservas (*reservations*): representa las reservas realizadas por los clientes. Campos:

- id
- user_id
- schedule_id
- estado (activa / cancelada)
- created_at
- updated_at

Relaciones principales

El sistema deberá implementar las siguientes relaciones:

- Un usuario puede tener muchas reservas
- Un horario puede tener muchas reservas
- Una clase puede tener varios horarios.
- Una clase pertenece a un entrenador
- Un entrenador puede impartir muchas clases

Funcionalidades de la aplicación

La aplicación debe incluir al menos las siguientes funcionalidades:

1. Autenticación

La aplicación deberá permitir:

- registro de usuarios
- inicio de sesión
- cierre de sesión

Al iniciar sesión se guardará en sesión información como: id_usuario, nombre y rol. Esta información se utilizará para mostrar diferentes opciones en la aplicación.

2. Dashboard dinámico

Después de iniciar sesión, cada usuario verá un panel distinto según su rol:

➤ **Dashboard *administrador*:** debe incluir acceso a:

- gestión de usuarios
- gestión de clases
- gestión de horarios
- gestión de reservas
- estadísticas del gimnasio

➤ **Dashboard *cliente*:** debe incluir:

- ver clases disponibles
- realizar reservas
- ver mis reservas
- cancelar reservas
- editar perfil

➤ **Dashboard *entrenador*:** debe incluir:

- ver clases que imparte
- ver alumnos inscritos
- ver su calendario de clases

3. Gestión de clases

El administrador podrá:

- crear clases
- editar clases
- eliminar clases
- ver listado de clases

Campos del formulario:

- nombre
- descripción
- duración
- imagen
- capacidad
- entrenador

4. Gestión de horarios

El administrador podrá asignar horarios a cada clase.

Ejemplo:

Clase: Yoga
Fecha: 10/05/2026
Hora inicio: 10:00
Hora fin: 11:00

5. Sistema de reservas

Los clientes podrán reservar plaza en un horario disponible. El sistema deberá controlar que no se supere la capacidad máxima de la clase.

Cuando se haga una reserva:

- se registrará en la tabla *reservations*
- el estado será *activa*

6. Cancelación de reservas

Los clientes podrán cancelar reservas. Cuando esto ocurra el estado de la reserva pasará a cancelada.

7. Mis reservas

Los clientes podrán ver una tabla con sus reservas. Ejemplo:

Clase | Fecha | Hora | Estado

8. Panel del entrenador

Los entrenadores podrán ver:

- ✓ las clases que imparten
- ✓ los horarios de cada clase
- ✓ los alumnos inscritos

9. Gestión de usuarios

El administrador podrá:

- ✓ ver todos los usuarios
- ✓ editar usuarios
- ✓ eliminar usuarios
- ✓ cambiar el rol de un usuario

10. Estadísticas

Debe existir una sección de estadísticas para el administrador. Ejemplos de estadísticas:

- número total de usuarios

- número total de clases
- clases con más reservas
- clientes con más reservas
- reservas por día

Organización de rutas

Las rutas deberán organizarse en grupos según el acceso:

- Rutas públicas
- Rutas para usuarios autenticados
- Rutas de administración

Control y seguridad

El sistema debe controlar quién tiene acceso:

- ✓ Verificar qué usuario ha iniciado sesión.
- ✓ Al panel de administrador se va a permitir acceso únicamente a usuarios con rol *admin*.
- ✓ Para el entrenador se va a permitir el acceso a funcionalidades específicas.

Modelos

Se deberán crear los modelos correspondientes y cada modelo debe definir:

- campos fillable
- relaciones con otros modelos

Migraciones

Se deberán crear migraciones para todas las tablas necesarias. Las migraciones deben incluir:

- claves primarias
- claves foráneas
- timestamps

Seeders

Se deben crear datos iniciales para facilitar las pruebas. Por ejemplo (considera poner más):

- 1 administrador (o varios)
- Varios entrenadores.
- 5 o más clientes.
- Varias clases.
- Algunos horarios.

Vistas Blade

La aplicación debe utilizar Blade para generar las vistas. Debe existir un layout principal que incluya:

- menú de navegación
- contenido dinámico
- mensajes flash

El menú debe adaptarse según el rol del usuario.

Validación de formularios

Todos los formularios deben validar los datos introducidos por el usuario. Ejemplos:

- nombre obligatorio
- duración numérica
- capacidad mayor que 0
- fecha válida

Uso de cookies

Se debe implementar funcionalidad que utilice cookies. Ejemplo:

- recordar usuario
- guardar preferencia de vista

Funcionalidades opcionales (para mejorar la nota)

Se pueden añadir mejoras como:

- búsqueda de clases
- filtrado por entrenador
- paginación de resultados
- calendario visual de clases

Resultado esperado

La aplicación final deberá permitir:

- registrarse e iniciar sesión
- consultar clases del gimnasio
- reservar plazas
- cancelar reservas
- gestionar clases y usuarios desde el panel de administración
- mostrar estadísticas básicas del sistema